Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Software Design Description (SDD)
Fault Logger

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	Thomas Marmeth Software Designer	2011-10-11
Verified :	Eric Malo, Eng. System Engineer	2011-10-11
Approved :	Martin Ouellet, Eng. Software Department Supervisor	2011-10-11

Document No.	**201111583**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-07**

Revision List

Rev.	ECD#	Date	By	Description of changes
1	---	2011-10-07	Thomas Marmeth	Released version
0.1	---	2011-10-03	Thomas Marmeth	Implemented comments from inspection I-1805
0	---	2011-08-25	Thomas Marmeth	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and IEEE Std 1558-2005, IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems approved on 2005-05-23.

Table of Contents

Revision List 2
Preface 3
Table of Contents 4
List of Figures 5
List of Tables 5

1 INTRODUCTION 6

1.1.1 Purpose 6
1.1.2 Scope 6
1.1.3 Definitions and Acronyms 6
1.1.4 Document Organization 8

2 REFERENCES 9

3 SOFTWARE DESIGN 10

3.1 Decomposition Description 10
3.1.1 Module Decomposition 10
3.1.2 Concurrent Process Decomposition 10
3.1.3 Data Decomposition 11
3.2 Dependency Description 14
3.2.1 Inter-Module Dependencies 14
3.2.2 Inter-Process Dependencies 14
3.2.3 Data Dependencies 14
3.3 Interface Description 15
3.3.1 Fault Logger functions 15
3.4 Detailed Design 18
3.4.1 Fault Logger Module Design 18
3.4.2 Data Entities Detailed Design 26

APPENDIX A 27

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 10
Figure 3-2: Inter-Process Dependencies Diagram 14
Figure 3-3: FLOG AssociatedEvent 18
Figure 3-4: FLOG CouterSet 18
Figure 3-5: FLOG Event Deletion 19
Figure 3-6: FLOG Directory Lookup 20
Figure 3-7: FLOG File Transfer 21
Figure 3-8: FLOG Recording 23
Figure 3-9: FLOG Reset LDTS 24
Figure 3-10: FLOG Send Error 25
Figure 3-11: FLOG SyncRequest 26

List of Tables

Table 3-1: Structure of Event log counter 11
Table 3-2: Event Set Reset Structure 11
Table 3-3: Event Log Index Structure 12
Table 3-4: Structure of Event Log 12
Table 3-5: Structure of Extra attributes (ExtraAttribute) 13
Table A-1: Default Attribute Setting 27

1 INTRODUCTION

This document is the Software Design Description (SDD) of the Fault Logger Module developed by Vapor Stone Rail Systems.

The Fault Logger Module can be part of any of the Door Module System (DMS) Applications.

1.1.1 Purpose

The purpose of this SDD is to describe the software components and implementation details of the Fault Logger Module.

The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.1.2 Scope

The Fault Logger Module is a software module of the Door Module System (DMS) Application. This module is not responsible of door operation.

1.1.3 Definitions and Acronyms

1.1.3.1 Definitions

Boot Loader	Software starting the Door Control Module application.
Car Network	Data bus and optionally trainlines connecting the Door Control Module with the rest of the train (others Door Control Modules in the car and others cars).
Communication Gateway	A specific processor based electronic module running a Communication Gateway Module Application. It acts as gateway between the Door Control Module Application and the Car Network/Portable Test Application.
Door Control Module	A specific processor based electronic module running a Door Control Module Application. It is used for controlling the door motor.
Door Control Unit	The final unit that is used to control one or more door panels. This unit is normally made of one or two door control modules running the Door Control Module Application and possibly one Communication Gateway Module Application.
Door Module System	A term that is used for generically identifying a processor based electronic module running any software within any Door Control Unit.
HW_COM	Communication interface used for communication between Portable Test Application and Door Control Module application, that can be USB, serial or Ethernet.

1.1.3.2 Event Level Definitions

Event:	Term used to globally identify an Info, an Exception or a Failure.
Info:	Info is an occurrence of an Informative condition monitored by the system, which is neither an exception nor a failure.
Exception:	An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).

Failure: A failure is an occurrence of a condition that is outside of the specified parameters and affecting this Line Replaceable Unit. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

1.1.3.3 Acronyms

ASCII	American Standard Code for Information Interchange
CDRL	Contract Data Requirements List
CL	CarLine
Cmd	Command
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DCU	Door Control Unit
DMS	Door Module System
ECD	Engineering Change Document
FLOG	Fault logger
FMP	Fault Management Plan
HW	Hardware
IEEE	Institute of Electrical and Electronics Engineers
IO	Input Output
KB	Kilobytes
LDTS	Local Diagnostic and Test System
LRU	Line Replaceable Unit
LS	Lock Switch
MHz	MegaHertz
ms	millisecond
NVRAM	Non-Volatile Random Access Memory
OC	OverCurrent
OS	Operating System
PSS	Position Sensor Switch
PTA	Portable Test Application
RAM	Random Access Memory
RS232	Recommended Standard 232 (Computer Serial Interface)
RTC	Real-Time Clock
RX	Receiver
SCI	Software Configuration Items
SDD	Software Design Description
SPI	Serial Peripheral Interface
SRS	Software Requirements Specification
Std	Standard
TS	Technical Specification
USB	Universal Serial Bus
VSRS	Vapor Stone Rail Systems
WIP	Work in Progress
w/o	Without

1.1.4 Document Organization

The Software Design Description (SDD) document describes the architectural software design of the DMS Application Fault Logger Module. This SDD is divided into 3 sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software Design summarizes followings:
    - Decomposition Description: describes software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the function interface for each entity.
    - Detailed Design: details the functions algorithms implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.

Within the document, diagrams, flowcharts and tables are used to present the Fault Logger Module software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- The diagram Figure 3-1 describes the dependencies between different modules.
- Tables contain data structures used by the Fault Logger Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2005	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

S17-07	Software Design Description Template, Rev. 0, 2002-01-15 complemented with requirements from IEEE Std 1558.
CMM-01	Software Engineering Process, Rev.3
CMMI-02	Software Engineering Policy Manual, Rev. 1
SACS	Software Applications Configuration Specifications, Project's specific.
201111489	Generic Software Documentation Project – Portable Test Interface Protocol, Interface Control Document (ICD), Rev. 01
FMP	Fault management Plan (FMP), Project's specific.

3 SOFTWARE DESIGN

3.1 Decomposition Description

Figure 3-1 gives an overview of the modules interacting with the Fault Logger Module.

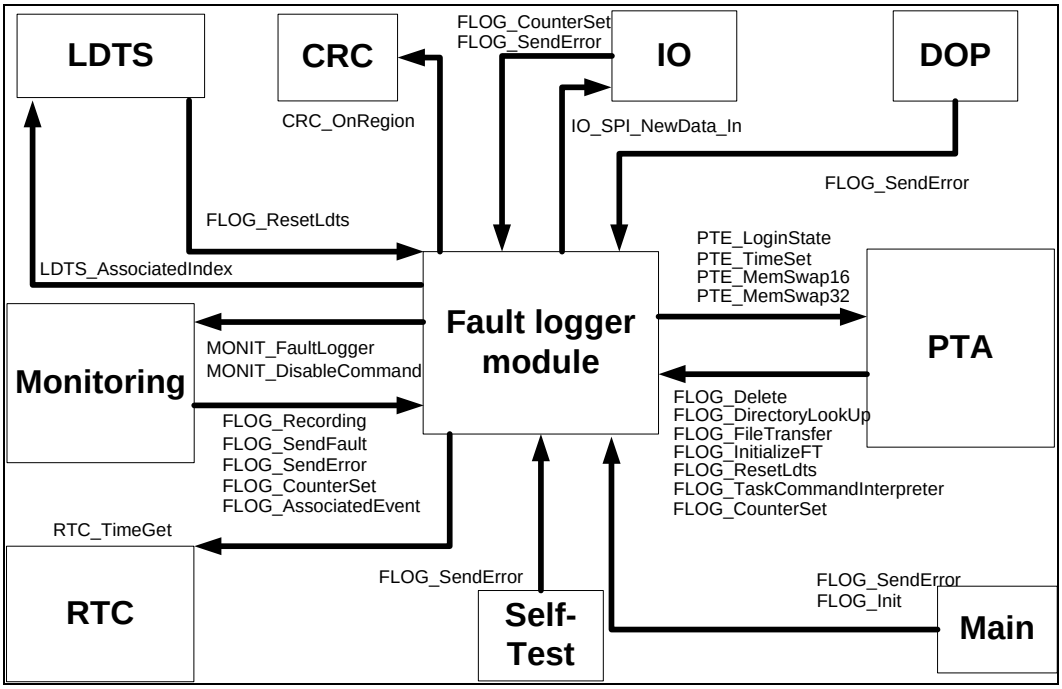

Figure 3-1: Inter-Module Dependencies Diagram

3.1.1 Module Decomposition

3.1.1.1 Fault Logger Module Description

The Fault Logger Module is a single entity which receives commands from various modules to save a specific fault. It manages the log file in the non-volatile memory and provides access for reading, writing and deleting faults. It updates the fault counters in the System Counters structure.

3.1.2 Concurrent Process Decomposition

3.1.2.1 Fault Logger Task Description

The Fault Logger module performs the following tasks:

- Event Recording
- Event Delete
- Event Directory Lookup
- Event File Transfer
- Event Log Sync Request

The module shall be linked with the final executable and do not use process or threading.The module functions are designed to support multi-threading.

3.1.3 Data Decomposition

3.1.3.1 Fault Logger Data Description

Table 3-1: Structure of Event log counter

Identification	EventLogCounter
Entity Type	struct CounterStruct

Prototype	`// Maximum number of fault in the system`
	`#define FAULT_MAX_NUMBER 256`
	
	`// Structure containing the number of event in a day`
	`typedef struct _CounterStruct`
	`{`
	`  unsigned long ulTimeStamp; // Time Stamp of the Event, 4 bytes`
	`  unsigned char ucEventCounter[FAULT_MAX_NUMBER]; // Counter`
	`  // associated with the fault number`
	`  unsigned int uiCrc; // CRC of the 2 previous members`
	`  // ulTimeStamp and ucEventCounter[]`
	`}CounterStruct;`
	
	`CounterStruct huge EventLogCounter;`

3.1.3.1.1 Event Set Reset structure

List of events in “fault” state.

Table 3-2: Event Set Reset Structure

Identification	EventSetReset
Entity Type	struct EventSetResetStruct

Prototype	`// Maximum number of fault in the system`
	`#define FAULT_MAX_NUMBER 256`
	
	`typedef struct _EventSetResetStruct`
	`{`
	`  unsigned char ucEvent[FAULT_MAX_NUMBER+1]; // One byte per fault`
	`  // ucEvent[x]== 0, event not set`
	`  unsigned int uiCrc; // CRC of ucEvent`
	`}EventSetResetStruct;`
	
	`EventSetResetStruct far EventSetReset;`

3.1.3.1.2 Event Log Index Structure

The event log index is used to know the status of the internal event log file.

Table 3-3: Event Log Index Structure

Identification	EventLogIndex
Entity Type	struct EventLogIndexStruct

Prototype	`typedef struct _EventLogIndexStruct`
	`{`
	`  unsigned int uiStatus;  // Status of the event log can be either:`
	`  // WIP_ENABLE  0x01 (bit 0 == true)`
	`  // ROLLOVER  0x02 (bit 1 == true)`
	`  // Bit 2..15 unused`
	`  unsigned int uiNextEvent;  // Next free event in the log`
	`  unsigned int uiWorkingEvent; // Working event Index`
	`  unsigned int uiCrc;  // CRC`
	
	`}EventLogIndexStruct;`
	
	`EventLogIndexStruct far EventLogIndex;`

3.1.3.1.3 Internal Event Log File Structure

Structure describes the Event Log File array stored in NVRAM.

Table 3-4: Structure of Event Log

Identification	EventLogFile
Entity Type	struct EventInternalLogRecordStruct

Prototype	`// Default Maximum number of fault in the fault logger`
	`// The maximum number of fault is configurable on a project per project basis.`
	`#define DEFAULT_EVENT_MAX_NUMBER  1500`
	
	`typedef struct _EventInternalLogRecordStruct`
	`{`
	`  unsigned long ulTimeStamp; // Time of the event, as recorded by the`
	`  // subsystem's clock. The time stamp is a`
	`  // extended "C" type 32 bits (unsigned)`
	`  // serial number that represent the number`
	`  // of seconds elapsed since January 1st`
	`  // 1970.`
	`  //In the case where a snapshot`
	`  // file is related to this event, this`
	`  // field also serves as the name of this`
	`  // file.`
	`  unsigned int uiEventCode;  // The event code.`
	`  unsigned int uiAttributes; // The attributes of the event.`
	`  // - Index of the Snapshot if there is one`
	`  // - The index of the old event related to`
	`  //  the snapshot if the snapshot is being`
	`  //  overwritten`
	`  // - The state of the fault either set or`
	`  // reset`
	`  unsigned int ucExtraAttribute;// Extra attributes of the event.`
	`  unsigned char ucSpare;  // Extra byte for structure alignment.`
	`  unsigned int uiCrc;  // CRC of the event log`
	
	`}EventInternalLogRecordStruct;`
	
	`EventInternalLogRecordStruct huge EventLogFile[EVENT_MAX_NUMBER];`

Table 3-5: Structure of Extra attributes (ExtraAttribute)

```
typedef struct _ExtraAttribStruct // Msbit first
{
   BIT_FIELD_STRUCT
   (
      Bit bType  :1,        // This bit field indicates the type of event:
                            // 0 = Pulse.
                            // 1 = Set / Reset.
      Bit bState :1,        // Indicates the state of the event: ( This field is
                            // irrelevant if the event type is "Pulse").
                            // 0 = Reset.
                            // 1 = Set.
      Bit bSyncReq :1,      // This bit field indicates if the event has been issued
                            // following a request:
                            // 0 = Event issued normally.
                            // 1 = Event issued following a request.
      Bit bSpare :1,        // This field is spare.
      Bit bFile  :4.        // This field is used to indicate the index of the
                            // snapshot file
    )
} ExtraAttribute;
```

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

3.2.1.1 Fault Logger Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the Fault Logger Module.

3.2.2 Inter-Process Dependencies

Figure 3-2 shows the Inter-Process Dependencies Diagram of the Fault Logger Module.

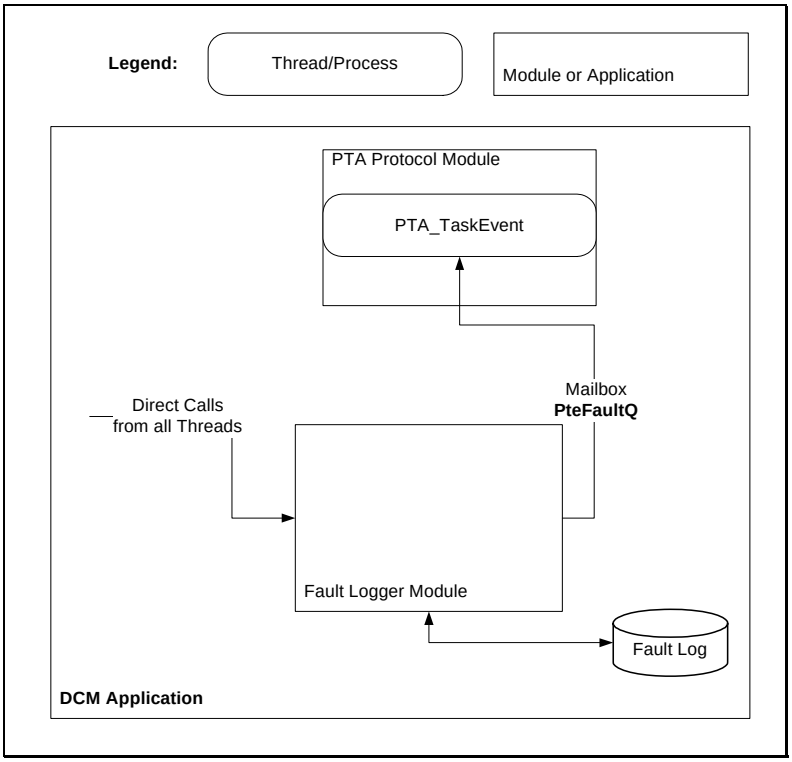

Figure 3-2: Inter-Process Dependencies Diagram

3.2.3 Data Dependencies

Not applicable.

3.3 Interface Description

This section describes the functions of the Fault Logger Module. The function name, a description of the function, input and output parameters required by the function are provided for each function.

3.3.1 Fault Logger functions

The Fault Logger module implements the following functions (sorted alphabetically):

3.3.1.1 Fault Logger External Interfaces

Function Name	unsigned int FLOG_AssociatedEvent(void)
Description	Returns the current position of the event log pointer.
Input	None
Output	None
Return	The current position of the event log pointer

Function Name	void FLOG_CounterSet(unsigned long ulTimeSec)
Description	Sets the new value of the timestamp of the fault counter.
Input	unsigned long ulTimeSec :Timestamp of the current day for the date and 00:00:00 AM for the time.
Output	None
Return	None

Function Name	void FLOG_Delete (void)
Description	Deletes the fault log.
Input	None
Output	None
Return	None

Function Name	unsigned long FLOG_DirectoryLookup (void)
Description	Gives the size of the fault log.
Input	None
Output	None
Return	Size of the Event Log

Function Name	unsigned char FLOG_FileTransfer (unsigned char xhuge *pucBuffer, unsigned int *puiBufferSize)
Description	Processes a fault log file transfer.
Input	unsigned char xhuge *pucBuffer : A pointer to the data destination. unsigned int *puiBufferSize : A pointer to the size of the data.
Output	Data copied Number of bytes copied into the buffer The function returns: FS_XFER_UNDERWAY while is not completed FS_XFER_OK when the transfer is completed FS_IO_ERR when a fault has been overridden
Return	None

Function Name	void FLOG_Init(void)
Description	Initializes the Fault Log module.
Input	None
Output	None
Return	None

Function Name	unsigned char FLOG_InitializeFT (void)
Description	Initializes buffer to get ready for File Transfer.
Input	None
Output	TRUE if everything's OK, FALSE if there's no event in the log
Return	None

Function Name	void FLOG_Recording (unsigned long ulTimeStamp, unsigned int uiEventCode, unsigned int uiAssociatedEvent, unsigned char ucSnapshotIndex, unsigned char ucState, unsigned int uiFaultExInfo)
Description	Records a fault in non-volatile memory.
Input	unsigned long ulTimeStamp : Time when the fault occurred unsigned int uiEventCode : Event code of the fault unsigned int uiAssociatedEvent : Associated Event of the Fault, unsigned char ucSnapshotIndex : Index of the Snapshot file unsigned char ucState : Contains either FAULT_SET or FAULT_RESET int uiFaultExInfo : Fault info
Output	None
Return	None

Function Name	void FLOG_ResetLdts (unsigned int uiAssociatedEvent)
Description	Resets the Snapshot Index of the specified event.
Input	unsigned int uiAssociatedEvent : Associated event related to the snapshot to delete
Output	None
Return	None

Function Name	void FLOG_SendError(unsigned int uiError, unsigned int uiExtra)
Description	Sends the error to the error queue and send a message to PTA transmission
Input	unsigned int uiError : Fault number part 1 (first 2 bytes) unsigned int uiExtra : Fault number part 2 (last 2 bytes)
Output	Error in error queue message
Return	None

Function Name	void FLOG_SendFault (unsigned int uiEventCode, unsigned char ucSnapshotIndex, unsigned char ucState, unsigned char ucSync)
Description	Posts an event in the fault queue and send a message to PTA transmission
Input	unsigned int uiEventCode : Event code of the fault unsigned char ucSnapshotIndex : Index of the Snapshot file unsigned char ucState : Contains either FAULT_SET or FAULT_RESET unsigned char ucSync : EVENT_SYNC_REQUEST or EVENT_SYNC_NO_REQUEST
Output	Fault in fault queue message
Return	None

Function Name	void FLOG_SyncRequest (unsigned char ucOrigin)
Description	Posts all the events that are in a set state in the fault queue.
Input	unsigned char ucOrigin : EVENT_SYNC_REQUEST or EVENT_SYNC_NO_REQUEST
Output	None
Return	None

3.4 Detailed Design

This section describes how the fault logger module works and what operations are performed.

3.4.1 Fault Logger Module Design

3.4.1.1 FLOG_AssociatedEvent

This function returns the current position of the Event log pointer EventLogIndex.uiNextEvent, and therefore to the next available position in the Event Log.

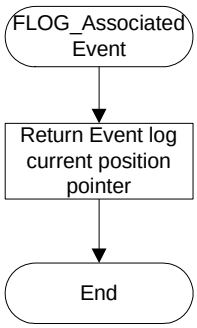

Figure 3-3: FLOG AssociatedEvent

3.4.1.2 FLOG_CounterSet

This function will reset the fault counter each day, and reset the fault timestamp. This function will be used when a new present time is set, or when a new fault has to be logged.

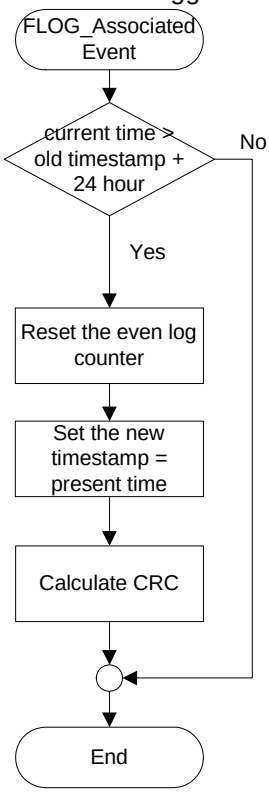

Figure 3-4: FLOG CouterSet

3.4.1.3 FLOG_Delete

This function will be called when the Fault Logger module receives a command to delete the event log file from the requesting module. The deletion of the log file is achieved by writing 0x00 in the timestamp field of each record in the file. There is no parameter received or returned to the caller. Figure 3-5 shows the sequence used to process the FLOG Event Delete.

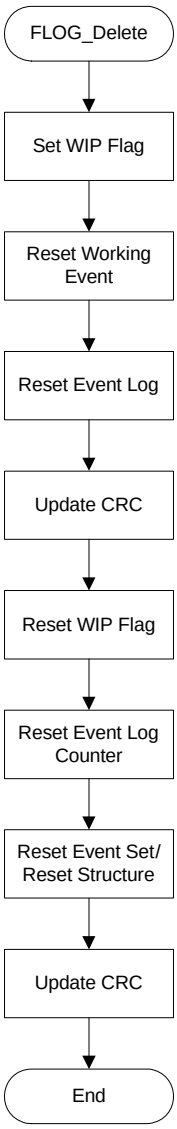

Figure 3-5: FLOG Event Deletion

3.4.1.4 FLOG_DirectoryLookup

The Fault Logger module receives from the requesting module a directory lookup command prior to a file transfer. There is no parameter received for directory lookup.
The module returns:

- The size of the data that are in the event log

In case the Event log file is empty, the size returned is 0. The module sets the pointers for the next file transfer request.
To calculate the size of the file, the module simulates the real packet format sent to the requesting module port.

Figure 3-6 shows the sequence used to process the FLOG Directory Lookup.

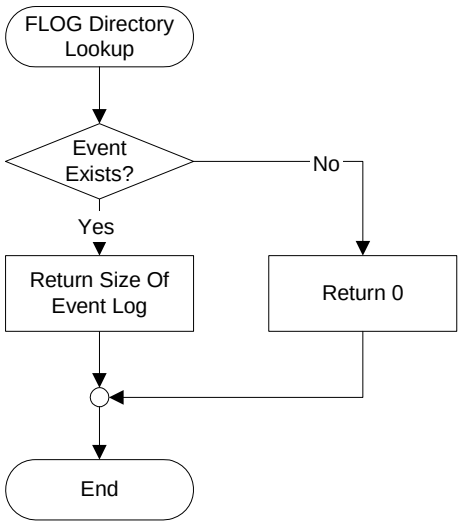

Figure 3-6: FLOG Directory Lookup

3.4.1.5 FLOG_FileTransfer

An Event Log File Transfer is used only when a file transfer is requested. The Fault Logger module receives from the requesting module:

- File transfer command with a pointer to a buffer where the data will be copied. The buffer is used to return formatted data.
- A pointer to the size of the filled buffer. It also contains the maximum number of characters to transmit.

The module returns the status of the file transfer either:

- FS_XFER_OK when the transfer is completed.
- FS_UNDERWAY when the transfer is underway.
- FS_IO_ERR when a fault to be transmitted has been overwritten.

The Module formats the data of the event as required in the ICD document #201111489.
The formatted data is copied in the location pointed by the pointer received. The pointer is post incremented and always points to the next available position. The module copies a maximum of 6 x 32bytes (192 bytes) of data on each request.

Figure 3-7 shows the sequence used to process the FLOG File Transfer.

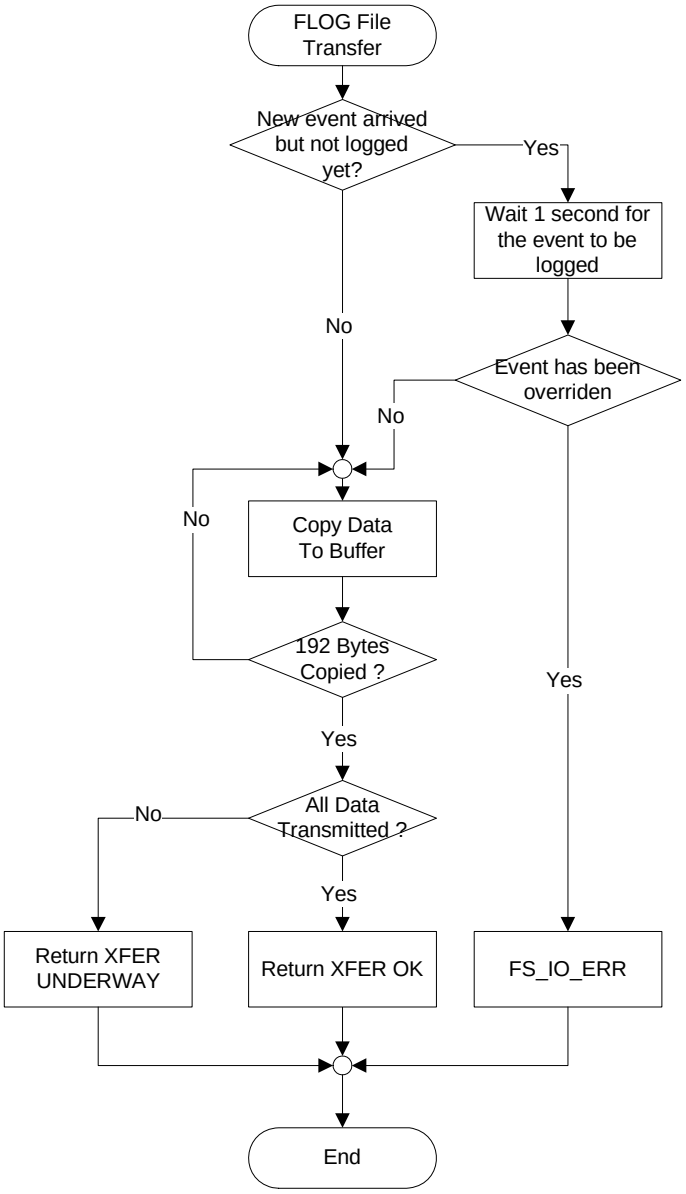

Figure 3-7: FLOG File Transfer

3.4.1.6 FLOG_Init

This function initializes the Fault Log module by resetting the module variable to zero and creating the messages queues.
This function is too simple to provide an algorithm/flowchart.

3.4.1.7 FLOG_InitializeFT

This function initializes the buffer to get ready for File Transfer, by calculating the size to transmit and initializing pointers at beginning of log.
This function is too simple to provide an algorithm/flowchart.

3.4.1.8 FLOG_Recording

The Event Log File can contain up to 1500 faults (configurable value) with a maximum of 10 occurrence of a fault per day. The log file is a circular buffer type of file that wraps around.
Knowing that each fault takes 12 bytes of memory space (the size of Event Internal Log Record structure), the total number of bytes required to log all faults is 1500 faults x 12 bytes/fault = 18000 bytes.
The module receives the following parameters

- The time of the event
- The event code to log
- The index of the Snapshot if there is one
- The index of the old event related to the snapshot if the snapshot is being overwritten
- The state of the fault either set or reset

No parameters are returned from this module.
The first thing done in this module is to indicate the state (either set or reset) of the event in a table. The structure of this table is shown in Table 3-2: Event Set Reset Structure:

- 1 Byte for each event code
- 2 Bytes for the CRC of the previous bytes of the structure.

A WIP flag is set to TRUE when a fault is written and then reset. This mechanism allows to keeps the integrity of the data and the event log file if a reset occurs while the data is written into log file. The roll over flag is used to indicate that the event log has been filled and newer data are written on oldest records. The next free event index always points to the next available record in the event log.
If there is a Snapshot that has been overwritten, the index of the old Snapshot is being deleted in the record of the event. During this process the Working Event index contains the index of the record being modified.

The Event Logger Structure is updated each time an event is written in the event log or a modification occurs in a record of the event log.

A simple structure is keeping track of the status of the event log file. It is described in Table 3-3: Event Log Index Structure. The beginning and ending address are fixed and known for the log file. The internal structure of the Event log is shown in Table 3-4: Structure of Event Log.

Figure 3-8 shows the sequence used to process the FLOG Recording.

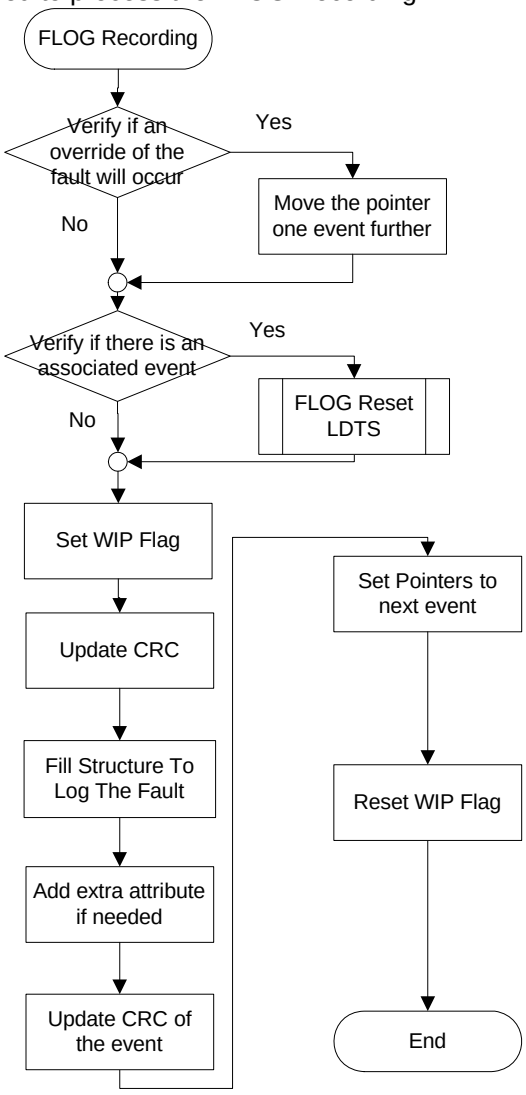

Figure 3-8: FLOG Recording

3.4.1.9 FLOG_ResetLdts

Figure 3-9 shows the sequence used to process the FLOG Reset LDTS.

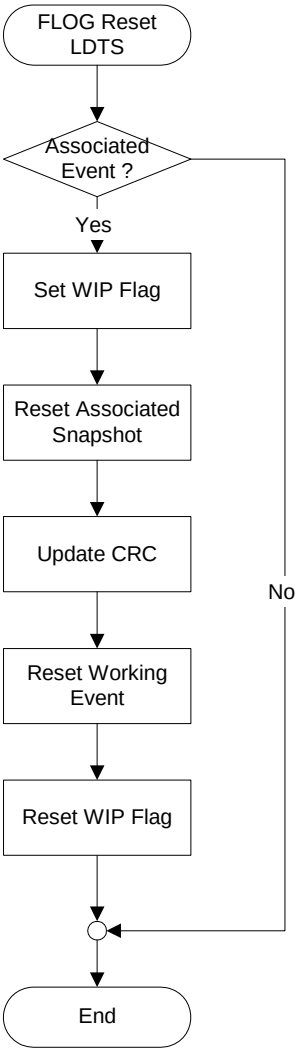

Figure 3-9: FLOG Reset LDTS

Snapshots files are attributed to some faults. If there is a snapshot file that is associated to a fault, the fault logger module maintains the extra attributes structure. The last 4 bits (LSbits) of this structure are reserved to indicate which snapshot is associated with the event recorded in memory.
There are 16 attributes that can be set for each fault recorded. The default attribute setting for each fault can be found in the FMP document.
Table A-1 lists the default attribute setting specific to the DCM Application.

3.4.1.10 FLOG_SendError

Figure 3-10 shows the sequence used to process the FLOG Send Error.

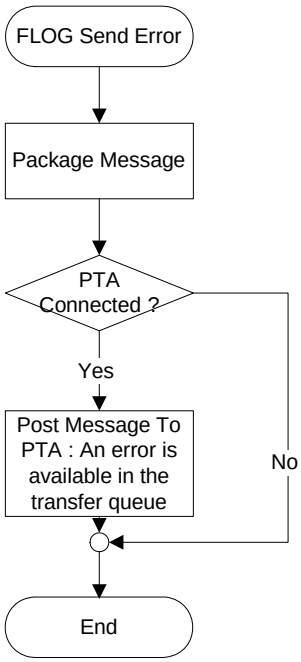

Figure 3-10: FLOG Send Error

3.4.1.11 FLOG_SendFault

This function posts an event in the fault queue. This function will also verify if there is a synchronization request from PTA. If this is the case, the function will post message to PTA transmission after having verified if the PTA is logged in before sending the message.
This function is too simple to provide an algorithm/flowchart.

3.4.1.12 FLOG_SyncRequest

The Event Log Synchronization request is used to resynchronize the events that are in set state. This module receives no parameters.
The module returns:

- The record is formatted as required in the ICD Document #201111489. The “aSyncReq” bit in “ExtraAttribute” field is set. The Data is returned via the Fault queue to be sent to the PTA.

Figure 3-11 shows the sequence used to process the FLOG Sync Request.

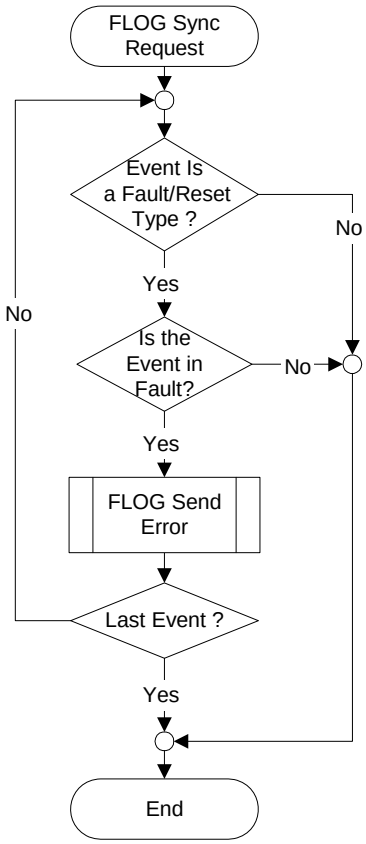

Figure 3-11: FLOG SyncRequest

3.4.2 Data Entities Detailed Design

Not applicable.

APPENDIX A

The data contained in this appendix is project's specific, the values given hereafter are the default values.

Table A-1 gives the defaut attribute setting for the DCM application:

Table A-1: Default Attribute Setting

Bit #	Attribute Name	Default Setting
0-1	Event (00_b) Failure (01_b) Exception (10_b) Reserved (11_b)	Event (00_b)
2	Archive	Used
3	Snapshot	Not used
4	Disable Open Command	Not Used
5	Disable Close Command	Not Used
6	Reduced Speed	Not Used
7	Spare Attribute	Not Used
8	Spare Attribute	Not Used
9	Spare Attribute	Not Used
10	Spare Attribute	Not Used
11	Spare Attribute	Not Used
12	Spare Attribute	Not Used
13	Spare Attribute	Not Used
14	Spare Attribute	Not Used
15	Spare Attribute	Not Used